Coding Assessment: BookNest

Deliverable: A GitHub repository link + a Loom/screen-recording demo (4 to 6 minutes).

Requirements

Build **BookNest**, a reading-tracker app where a user manages books, organizes them into custom shelves, shares shelves with other users under different permission levels, logs reading progress, and can lend books to other users on the platform.

This has real relationships, real state, and real edge cases. Read all of it before you start coding, and plan your data model first.

Core scope (must build)

Authentication and accounts

1. Sign up with name, email, password. Validate email format and enforce password rules you define and document.
2. Log in / log out using **JWT with a refresh-token flow**: a short-lived access token and a longer-lived refresh token. Document how you store each and why.
3. Passwords hashed with bcrypt or argon2. **Plaintext is an automatic fail.**
4. A user can only see and modify their own data, except where lending or a shared-shelf role explicitly allows it (see below). Enforce all of this on the backend. We will call your API directly to test it.
5. Unauthenticated requests to protected endpoints return 401. Expired access token returns 401 and the client transparently refreshes and retries.

Books 6. Add a book: title, author, status (Want to Read, Reading, Finished), total pages, optional rating (1 to 5), optional notes. 7. List, edit, delete. 8. Combined filter (by status) AND search (title/author) active together. 9. **Server-side pagination AND sorting** (by rating, title, or date added). The frontend sends page, filters, search, and sort; the backend does the work. No loading-everything-then-slicing.

Shelves (many-to-many) 10. Create custom shelves. A book belongs to many shelves; a shelf holds many books. Model the relationship properly. 11. View books on a shelf. Deleting a shelf must not delete its books; deleting a book must clean it out of all shelves with no orphaned references.

Shared shelves and roles (RBAC) 12. The creator of a shelf is its **owner**. The owner can share a shelf with other registered users (by email), granting each one of two roles: - **editor**: can add and remove books on that shelf - **viewer**: can see the shelf and its books but cannot change anything 13. Enforce these roles on the backend, not just by hiding buttons. A viewer who calls the "add book to shelf" API directly must be rejected. We will test this. 14. Only the owner can share the shelf, change someone's role, remove a collaborator, or delete the shelf. An editor or viewer attempting any of these gets a clear error. 15. A "Shared with me" view lists shelves other users have shared with the current user, showing their role on each. 16. Removing a collaborator (or deleting a shelf) must clean up cleanly: no orphaned share records, and the books themselves are never deleted.

Reading progress (validation + derived data) 17. Log current page for a "Reading" book; show progress percentage. 18. Validation: reject page > total pages, reject negatives, reject progress when total pages is unset, with clear error messages and no crashes. 19. When current page reaches total pages, status auto-updates to "Finished" and a "finished date" is recorded.

Lending (the hard feature: cross-user state and rules) 20. A user can mark one of their books as "lent to" another registered user (by that user's email). 21. The borrower sees, in a "Borrowed from others" view, books currently lent to them (read-only, they cannot edit the owner's book). 22. A book that is currently lent cannot be lent to a second person at the same time. Enforce this; return a clear error. 23. The owner can mark the book "returned", which clears the lending and makes it fully theirs again. 24. You cannot lend a book to yourself, and you cannot lend a book that does not exist or is not yours. Handle these cleanly.

Activity log 25. Record key events: book added, status changed, book lent, book returned, shelf shared, collaborator role changed/removed. Show a simple reverse-chronological activity feed on the dashboard.

Real-time updates (WebSockets) 26. Use WebSockets (Socket.io, native ws, or your framework's equivalent) for live updates. Polling does not satisfy this requirement; document your choice in the README. 27. When an owner lends a book to a borrower, the borrower's "Borrowed from others" view updates live, no refresh. When the owner marks it returned, it disappears from the borrower's view live. 28. When an editor adds or removes a book on a shared shelf, the other collaborators currently viewing that shelf see the change live. 29. The activity feed on the dashboard updates live as new events happen for that user. 30. The socket must be authenticated, and a user must only receive events meant for them: their own events, plus events for shelves shared with them. A viewer must not receive events for shelves they have no access to. Explain in the README how you secured the socket and how you scope events to the right people (not a global broadcast everyone hears). 31. Handle the disconnect case sensibly: if the socket drops, the app should still work (the user can refresh to get current state) and reconnect when possible. Describe your approach in the README.

Dashboard 32. Show for the logged-in user: counts by status, books finished this year, average rating, the shelf with the most books, how many books they currently have lent out, how many shelves are shared with them, and the recent activity feed.

Frontend quality requirements (not optional)

These apply across the whole app, we will check them:

- Every data-loading view shows a **loading state**, and a **clear error state** when a request fails (not a blank screen or a console error).
- Forms show **validation errors inline** (e.g., "Page cannot exceed total pages"), not just an alert or a crash.
- Buttons that trigger requests are disabled / show progress while the request is in flight (no double-submit).

Stretch goals (optional, only if core is rock solid)

Pick at most one or two.

- A few automated tests on critical paths (auth, lending rules, progress validation).
- Dockerized with docker-compose (frontend + backend + DB, one command).
- CSV import of books.
- Email (mock/console is fine) to the borrower when a book is lent to them.
- Optimistic UI updates with rollback on failure.
- Deploy it live (Vercel / Render / Railway / Netlify free tiers).

Tech you can use

The app must have a real backend and a real database.

- **Frontend:** Next.js / React / Vue.js
- **Backend:** Nest.js, OR Python + FastAPI/Flask/Django.
- **Database:** MongoDB, PostgreSQL, or MySQL. (SQLite only if you justify it.)
- **Auth:** JWT with refresh tokens (required, see item 2).
- **Real-time:** WebSockets (required, see the real-time section).
- **Styling:** Anything usable and intentional.

About using AI tools (read this honestly)

You may use ChatGPT, Claude, Copilot, or any AI tool. We use them too. We are not trying to catch you using AI.

The deal: **in the follow-up call we will pick parts of your code, ask you to explain them, make a small live change, and debug something.** This task has enough moving parts (refresh tokens, many-to-many, cross-user lending rules, validation, activity log) that copy-pasted code you do not understand will not survive questioning.

Honest advice:

- Use AI to learn and unblock, not to think for you.
- Read and understand everything before submitting. If AI wrote something you cannot explain, learn it or remove it.
- "I used AI here, here is what I learned and changed" scores positively.

What to submit

1. GitHub repository (public)

- Run instructions that work on a clean clone.
- .env.example with required variables (no real secrets).
- A seed script creating at least two test users with sample books, shelves, at least one shelf shared with the other user (one as editor, and ideally one as viewer), and at least one active lending, so we can test the sharing and borrow flows immediately.
- Granular commit history. Commit as you build.

2. README.md

Include:

- What the app does (short paragraph).
- How to run it (step by step, clean-clone tested).
- **Your data model:** collections/tables, and how users, books, shelves, shelf-shares/roles, and lending relate. Diagram or clear text.
- Stack choice and why.
- **How your refresh-token flow works** (what is stored where, what happens on expiry).
- **How you enforce shelf roles (owner/editor/viewer)** on the backend, and how you keep a viewer from making changes via a direct API call.
- **How your WebSocket setup works:** how you authenticate the socket, how you make sure a user only receives events meant for them (own events plus shelves shared with them), and how you handle disconnects/reconnects.
- What was hard and how you worked through it.

- Known issues / what is incomplete.
- What you would improve with more time.
- Where you used AI and what you learned.

3. Demo video (4 to 6 minutes)

- Show, with two browser windows side by side: sign up two users, add books and shelves, share a shelf with the second user as editor (and show a viewer cannot edit), have the editor add a book and watch it appear live in the owner's shelf view, log progress to auto-finish, lend a book to the second user and show it appear live in their borrowed view, mark returned and show it disappear live, and the dashboard + activity feed.
- Talk through how it works. Casual is fine.